

API Testing & Rest Assured Interview Questions & Answers

Complete Guide for Freshers to Experienced Professionals

Updated: April 27, 2026

Table of Contents

1. Fundamentals of API Testing (10 Questions)
2. REST APIs & HTTP Methods (8 Questions)
3. REST Assured Basics (9 Questions)
4. Authentication & Security (8 Questions)
5. Testing Strategies & Tools (7 Questions)
6. Performance & Advanced Topics (8 Questions)

1. Fundamentals of API Testing

Q1: What is API Testing and why is it important?

API Testing is the process of validating Application Programming Interfaces (APIs) by sending requests and checking if responses are correct, secure, and perform well. Unlike UI testing which checks appearance, API testing verifies backend operations and data exchange between systems.

Why Important:

- Catches bugs early before UI testing
- Faster test execution
- Tests backend logic directly
- Ensures secure communication
- Reduces costs compared to UI testing

Q2: What is an API and what are its main types?

An API is a set of rules that allows different software applications to communicate and exchange data. It acts as a bridge between different systems.

Main Types:

- **Web APIs:** Work over internet using HTTP (REST, SOAP)
- **Local APIs:** Allow apps within same system to communicate
- **Database APIs:** Allow applications to access databases
- **Operating System APIs:** Allow programs to use OS services

Q3: What is the difference between REST and SOAP APIs?

REST (Representational State Transfer) and SOAP (Simple Object Access Protocol) are two different architectural styles for building web services.

REST API:

- Uses HTTP methods (GET, POST, PUT, DELETE)
- Returns JSON or XML
- Stateless
- Lighter weight and faster

SOAP API:

- Uses XML messaging protocol
- More secure with WS-Security
- Stateful
- Better for complex operations

Q4: What are HTTP methods and their purposes?

HTTP methods define what action should be performed on a resource. The main methods used in API testing are:

Common HTTP Methods:

- **GET:** Retrieve data from server (idempotent, safe)

- **POST:** Create new data on server
- **PUT:** Update entire resource
- **PATCH:** Partially update resource
- **DELETE:** Remove resource
- **OPTIONS:** Describe communication options

Q5: What is an API Endpoint?

An API endpoint is a specific URL or route in an API that accepts requests and returns responses. It represents a specific resource or function that the API provides.

Example:

GET /api/users/123

Here:

- /api/users/123 is the endpoint
- GET is the HTTP method
- /api/users = base path
- /123 = path parameter (user ID)

Q6: What are Query Parameters and Path Parameters?

These are ways to pass data to an API endpoint.

Query Parameters: Appended after ? in URL

GET /api/users?page=2&limit=10

Path Parameters: Part of the URL path

GET /api/users/{userId}

GET /api/users/123

Q7: What is API Authentication and why is it important?

API Authentication verifies the identity of the user or system requesting access to the API. It ensures only authorized users can access protected resources.

Common Authentication Methods:

- **API Keys:** Simple key in header
- **Basic Auth:** Username:password in Base64
- **OAuth 2.0:** Token-based authorization
- **JWT:** JSON Web Tokens for stateless auth

Q8: What are HTTP Status Codes and their categories?

HTTP Status Codes indicate the result of an API request. They help testers understand whether the request succeeded or failed.

Status Code Categories:

- **2xx (Success):** 200 OK, 201 Created, 204 No Content

- **3xx (Redirect):** 301 Moved, 302 Found
- **4xx (Client Error):** 400 Bad Request, 401 Unauthorized, 404 Not Found
- **5xx (Server Error):** 500 Internal Server Error, 503 Unavailable

Q9: What is a Request Body and Response Body?

Request and Response bodies contain the actual data being sent and received.

Request Body: Data sent to server

```
POST /api/users  
{ "name": "John", "email": "john@example.com" }
```

Response Body: Data returned by server

```
{ "id": 1, "name": "John", "email": "john@example.com" }
```

Q10: What are Headers in API requests?

Headers provide metadata about the request or response. They contain additional information like authentication, content type, and caching instructions.

Common Headers:

- **Content-Type:** application/json
- **Authorization:** Bearer token
- **Accept:** Specifies expected response format
- **User-Agent:** Client information

2. REST APIs & HTTP Methods

Q1: Explain the difference between GET and POST requests

GET and POST are two fundamental HTTP methods with different purposes and behaviors.

GET Request:

- Retrieves data from server
- Parameters visible in URL
- No body (or minimal)
- Idempotent (safe to repeat)
- Used for fetching data

POST Request:

- Sends data to create resource
- Data in request body
- Non-idempotent
- Used for creating/modifying data

Q2: What is the purpose of PUT and PATCH requests?

Both PUT and PATCH are used to update resources, but they work differently.

PUT Request: Replaces entire resource

PUT /api/users/1

```
{ "name": "John", "email": "john@new.com", "age": 30 }
```

PATCH Request: Updates only specified fields

PATCH /api/users/1

```
{ "email": "john@new.com" }
```

Q3: What is API Versioning and why is it needed?

API Versioning allows developers to maintain multiple versions of an API to ensure backward compatibility when making changes.

Versioning Methods:

- **URL Path:** /api/v1/users vs /api/v2/users
- **URL Query:** /api/users?version=2
- **Header:** Accept: application/vnd.api.v2+json

Why Important: Prevents breaking existing client applications

Q4: What is a Mock API and when is it used?

A Mock API simulates the behavior of a real API by providing predefined responses without calling the actual backend service.

When Used:

- API is still under development
- External API is unavailable/unreliable

- Need to test edge cases

Tools: WireMock, Mockoon, MockServer

Q5: What is API Contract Testing?

API Contract Testing verifies that the API interaction between consumer and provider follows an agreed contract/specification.

Benefits:

- Detects breaking changes early
- Ensures API consistency
- Allows independent team development
- Prevents integration issues

Q6: What is the difference between Functional and Non-Functional API Testing?

Functional testing checks if API works correctly. Non-functional testing checks how well it performs under various conditions.

Functional Testing: Correct response data, proper error handling

Non-Functional Testing:

- Performance: Response time
- Load: Handle multiple requests
- Security: Vulnerabilities
- Reliability: Consistent behavior

Q7: What is Negative Testing in API Testing?

Negative testing validates how the API handles invalid or unexpected inputs. It ensures proper error handling.

Examples:

- Send null values
- Send wrong data types
- Send special characters
- Missing required fields
- Invalid authentication tokens

Q8: What is API Chaining?

API Chaining means using the response data from one API call as input for another API call. Useful for testing workflows.

Example:

1. POST /users (create user) → Response: id=123
2. GET /users/123 (fetch created user)
3. PUT /users/123 (update user)

4. DELETE /users/123 (delete user)

3. REST Assured Basics

Q1: What is REST Assured and why use it?

REST Assured is a Java-based library that simplifies testing REST APIs. It provides a readable syntax for sending requests and validating responses without complex HTTP client setup.

Advantages:

- Simple, readable syntax
- Supports all HTTP methods
- Easy response validation
- JSON/XML parsing support
- Works with TestNG/JUnit

Q2: How do you set up REST Assured in a project?

Add REST Assured dependency to Maven or Gradle build configuration.

Maven pom.xml:

```
<dependency>
<groupId>io.rest-assured</groupId>
<artifactId>rest-assured</artifactId>
<version>5.4.0</version>
</dependency>
```

Q3: Explain given(), when(), and then() methods

These three methods form the basis of REST Assured syntax, making tests readable and structured.

Structure:

given() → Setup request (headers, params, body)
when() → Perform action (HTTP method)
then() → Validate response (assertions)

Example:

```
given().header("Content-Type", "application/json")
.when().get("/api/users")
.then().statusCode(200);
```

Q4: How do you send a simple GET request?

Simple GET request to retrieve data from an endpoint.

Code:

```
given()
.when()
.get("https://api.example.com/users")
.then()
.statusCode(200);
```

Q5: How do you send a POST request with JSON payload?

POST request to create a new resource with data in the request body.

```
Code:
given()
.header("Content-Type", "application/json")
.body("{\"name\": \"John\", \"email\": \"john@example.com\"}")
.when()
.post("/api/users")
.then()
.statusCode(201);
```

Q6: How do you handle Path Parameters in REST Assured?

Path parameters are part of the URL path, used to identify specific resources.

```
Code:
given()
.pathParam("userId", 123)
.when()
.get("/api/users/{userId}")
.then()
.statusCode(200);
```

Q7: How do you handle Query Parameters in REST Assured?

Query parameters are appended to the URL, used for filtering or pagination.

```
Code:
given()
.queryParam("page", 2)
.queryParam("limit", 10)
.when()
.get("/api/users")
.then()
.statusCode(200);
```

Q8: How do you extract data from response?

Extract values from API response for further validation or use in subsequent requests.

```
Code:
Response response = given()
.when().get("/api/users/1")
.then().extract().response();

String name = response.jsonPath().getString("name");
int id = response.jsonPath().getInt("id");
```

Q9: What is a POJO and how is it used in REST Assured?

A POJO (Plain Old Java Object) is a simple Java class used to map JSON data. Makes request/response handling easier.

Example POJO:

```
public class User {  
    private String name;  
    private String email;  
    // getters and setters  
}
```

Usage:

```
User user = response.as(User.class);
```

4. Authentication & Security

Q1: What are the common authentication methods in API testing?

Authentication verifies the identity of the user or system requesting API access. Different methods provide varying levels of security.

Methods:

- **API Keys:** Simple key in header
- **Basic Auth:** Username and password (Base64 encoded)
- **OAuth 2.0:** Token-based authorization
- **JWT:** JSON Web Tokens (self-contained)

Q2: How do you test Basic Authentication in REST Assured?

Basic Authentication sends username and password encoded in Base64 format in the Authorization header.

Code:

```
given()
  .auth().basic("username", "password")
  .when()
  .get("/api/secure-data")
  .then()
  .statusCode(200);
```

Q3: How do you test OAuth 2.0 authentication?

OAuth 2.0 uses tokens for authorization. First get a token, then use it to access protected resources.

Code:

```
given()
  .auth().oauth2("access_token_value")
  .when()
  .get("/api/data")
  .then()
  .statusCode(200);
```

Q4: How do you test API Key authentication?

API Keys are simple tokens passed in headers to authenticate requests.

Code:

```
given()
  .header("X-API-Key", "your_api_key")
  .when()
  .get("/api/data")
  .then()
  .statusCode(200);
```

Q5: What is API Security Testing?

Security testing identifies vulnerabilities and weaknesses in APIs to ensure they are protected against attacks.

Security Concerns:

- Unauthorized access
- SQL Injection attacks
- XSS (Cross-Site Scripting)
- Data leaks
- Rate limiting bypass

Q6: What is Cross-Site Request Forgery (CSRF) and how to prevent it?

CSRF is an attack that forces a user to perform unwanted actions. Prevention requires implementing security mechanisms.

Prevention Methods:

- CSRF Tokens: Unique tokens per request
- SameSite Cookies: Restrict cookie usage
- Referrer Validation: Check request origin
- Double-Submit Cookies

Q7: What is Input Validation and why is it important?

Input validation ensures that user inputs are correct, secure, and meet expected criteria before processing.

Importance:

- Prevents SQL Injection
- Prevents XSS attacks
- Ensures data integrity
- Protects against malicious input

Q8: How do you handle rate limiting in API tests?

Rate limiting prevents abuse by restricting the number of requests. APIs return 429 (Too Many Requests) when limit exceeded.

Handling:

- Check for 429 status code
- Implement retry logic
- Add delays between requests
- Respect Retry-After header

5. Testing Strategies & Tools

Q1: What is Postman and how is it used in API testing?

Postman is a tool for designing, testing, and documenting APIs. It provides a user-friendly interface for sending requests and analyzing responses.

Features:

- Send HTTP requests
- Validate responses
- Create test scripts
- API documentation
- Organize requests in collections

Q2: How do you validate response body in REST Assured?

Validate specific fields in the response body to ensure API returns correct data.

Code:

```
.then()  
.statusCode(200)  
.body("name", equalTo("John"))  
.body("email", equalTo("john@example.com"))  
.body("age", greaterThan(18));
```

Q3: What is Boundary Value Analysis (BVA) in API testing?

BVA tests edge cases and boundary values of input ranges to find defects at boundaries.

Example:

Testing age field:

- Valid: 1, 18, 65, 120
- Boundary: 0, 1, 120, 121
- Invalid: -1, 150, 999

Q4: What is Data-Driven Testing in API testing?

Data-driven testing uses multiple sets of test data to run the same test with different inputs.

Approach:

- Store test data in CSV/Excel/JSON
- Use @DataProvider in TestNG
- Run same test with different values
- Better coverage and efficiency

Q5: What tools are available for API testing?

Various tools available for different types of API testing.

Popular Tools:

- **Postman:** Manual and automated testing
- **REST Assured:** Java-based automation
- **JMeter:** Load and performance testing
- **SoapUI:** SOAP and REST testing
- **Katalon:** All-in-one testing tool

Q6: How do you test APIs with different response formats?

APIs can return JSON, XML, or other formats. Use appropriate parsing methods.

JSON Response:

```
String value = response.jsonPath().getString("fieldName");
```

XML Response:

```
String value = response.xmlPath().getString("root.element");
```

Q7: What is API Documentation and why is it important?

API Documentation describes API endpoints, parameters, authentication, and expected responses. Crucial for testers and developers.

Should Include:

- Endpoint descriptions
- Request/response formats
- Authentication methods
- Status codes
- Error handling
- Rate limits

6. Performance & Advanced Topics

Q1: What is API Performance Testing and what does it measure?

Performance testing evaluates API speed, scalability, and reliability under different load conditions.

Metrics Measured:

- Response Time: How long API takes to respond
- Throughput: Requests handled per second
- Error Rate: Percentage of failed requests
- CPU/Memory Usage: Resource consumption

Q2: What is Load Testing and how is it different from Stress Testing?

Both are performance tests but with different goals.

Load Testing: Tests API under expected/peak load

Stress Testing: Tests API beyond expected load to find breaking point

Example:

Load: 1000 concurrent users

Stress: 10,000 concurrent users

Q3: What is API Monitoring and why is it important?

API Monitoring continuously tracks API availability, performance, and functionality in production.

Monitors:

- Uptime/Downtime
- Response times
- Error rates
- Traffic patterns
- Security threats

Q4: What is API Virtualization?

API Virtualization creates a virtual copy of an API that simulates the production API's behavior for testing purposes.

Benefits:

- Test before API is built
- Isolate dependencies
- Simulate unavailable APIs
- Reduce testing costs

Q5: How do you test APIs with large JSON responses?

For large or complex JSON responses, use efficient parsing and validation techniques.

Approach:

- Use JSONPath for specific field validation
- Deserialize to POJO objects
- Validate structure (schema validation)
- Test with different payload sizes

Q6: What is Response Schema Validation?

Schema validation ensures the response structure matches the expected format (JSON Schema or OpenAPI spec).

Example with JSON Schema:

```
.then()  
.body(JsonSchemaValidator.matchesJsonSchemaInClasspath("user-schema.json"))
```

Q7: How do you handle flaky tests in API testing?

Flaky tests pass and fail intermittently. Various strategies can reduce flakiness.

Solutions:

- Add proper waits (not hard sleeps)
- Implement retry logic
- Isolate test data
- Fix timing issues
- Use stable test environments

Q8: What is API Test Framework and what should it include?

A framework provides reusable components and best practices for organized API testing.

Should Include:

- Base utility classes
- Request/Response handling
- Test data management
- Error handling
- Logging and reporting
- Configuration management

Key Takeaways

- API testing is essential for modern software quality assurance
- REST Assured simplifies API automation testing with readable syntax
- Authentication and security testing are critical for API safety
- Performance testing ensures APIs scale under load
- Proper test organization and framework design improve maintainability
- API documentation should be comprehensive and accurate
- Combining multiple testing strategies provides comprehensive coverage

Good luck with your API testing interviews!